

Quickly reading very large tables as dataframes

Asked 13 years, 6 months ago Modified 1 year, 5 months ago Viewed 271k times  Part of [R Language Collective](#)



561



I have very large tables (30 million rows) that I would like to load as a dataframes in R. `read.table()` has a lot of convenient features, but it seems like there is a lot of logic in the implementation that would slow things down. In my case, I am assuming I know the types of the columns ahead of time, the table does not contain any column headers or row names, and does not have any pathological characters that I have to worry about.

I know that reading in a table as a list using `scan()` can be quite fast, e.g.:

```
datalist <- scan('myfile', sep='\t', list(url='', popularity=0, mintime=0, maxtime=0)))
```

But some of my attempts to convert this to a dataframe appear to decrease the performance of the above by a factor of 6:

```
df <-  
as.data.frame(scan('myfile', sep='\t', list(url='', popularity=0, mintime=0, maxtime=0)))
```

Is there a better way of doing this? Or quite possibly completely different approach to the problem?

[r](#) [import](#) [dataframe](#) [r-faq](#)

[Share](#) [Improve this question](#) [Follow](#)

edited Jun 3, 2018 at 12:36



[Uwe Keim](#)

39.3k

56

174

291

asked Nov 13, 2009 at 7:53



[eytan](#)

5,935

3

19

11

12 Answers

Sorted by: Highest score (default)



An update, several years later

492

This answer is old, and R has moved on. Tweaking `read.table` to run a bit faster has precious little benefit. Your options are:

1. Using `vroom` from the tidyverse package `vroom` for importing data from csv/tab-delimited files directly into an R tibble. See [Hector's answer](#).
2. Using `fread` in `data.table` for importing data from csv/tab-delimited files directly into R. See [mnel's answer](#).
3. Using `read_table` in `readr` (on CRAN from April 2015). This works much like `fread` above. The *readme* in the link explains the difference between the two functions (`readr` currently claims to be "1.5-2x slower" than `data.table::fread`).
4. `read.csv.raw` from [iotoools](#) provides a third option for quickly reading CSV files.
5. Trying to store as much data as you can in databases rather than flat files. (As well as being a better permanent storage medium, data is passed to and from R in a binary format, which is faster.) `read.csv.sql` in the `sqldf` package, as described in [JD Long's answer](#), imports data into a temporary SQLite database and then reads it into R. See also: the `RODBC` package, and the reverse depends section of the [DBI package](#) page. `MonetDB.R` gives you a data type that pretends to be a data frame but is really a MonetDB underneath, increasing performance. Import data with its `monetdb.read.csv` function. `dplyr` allows you to work directly with data stored in several types of database.
6. Storing data in binary formats can also be useful for improving performance. Use `saveRDS` / `readRDS` (see below), the `h5` or `rhd5` packages for HDF5 format, or `write_fst` / `read_fst` from the `fst` package.

The original answer

There are a couple of simple things to try, whether you use `read.table` or `scan`.

1. Set `nrows = the number of records in your data` (`nmax` in `scan`).
2. Make sure that `comment.char=""` to turn off interpretation of comments.
3. Explicitly define the classes of each column using `colClasses` in `read.table` .
4. Setting `multi.line=FALSE` may also improve performance in `scan`.

If none of these thing work, then use one of the [profiling packages](#) to determine which lines are slowing things down. Perhaps you can write a cut down version of `read.table` based on the results.

The other alternative is filtering your data before you read it into R.

Or, if the problem is that you have to read it in regularly, then use these methods to read the data in once, then save the data frame as a binary blob with `save` `saveRDS` , then next time you can retrieve it faster with `load` `readRDS` .

Share Improve this answer Follow

edited Sep 7, 2020 at 2:19

 **Hector Haffenden**
1,350 10 25

answered Nov 13, 2009 at 10:35

 **Richie Cotton**
117k 46 244 359

-
- 4 Thanks for the tips Richie. I did a little testing, and it seems that the performance gains with using the `nrow` and `colClasses` options for `read.table` are quite modest. For example, reading a ~7M row table takes 78s without the options, and 67s with the options. (note: the table has 1 character column, 4 integer columns, and I read using `comment.char=""` and `stringsAsFactors=FALSE`). Using `save()` and `load()` when possible is a great tip - once stored with `save()`, that same table takes only 12s to load. – [eytan](#) Nov 14, 2009 at 6:00
-
- 2 The "feather" package has a new binary format that plays nice with Python's pandas data frames – [rsoren](#) Mar 30, 2016 at 9:53 
-
- 5 I think maybe you need to update your post again with regards to the package `feather`. For reading data `feather` is much faster than `fread`. For example on a 4GB dataset I just loaded `read_feather` was about 4.5 times faster than `fread`. For saving data `fwrite` is still faster. blog.dominodatalab.com/the-r-data-i-o-shootout – [Z boson](#) Dec 8, 2016 at 10:42 
-
- 4 But the file sizes are much larger for feather than with RDS. I don't think it supports compression. The RDS file is 216 MB and the feather file is 4GB. So `feather` is faster for reading but it uses a lot more storage space. – [Z boson](#) Dec 8, 2016 at 10:48 
-
- 1 @Zboson If you need to store data frame in a file that can be accessed from both R and Python, then `feather` is a good option. If you only care about being able to read your data in R, `rds` is preferable. – [Richie Cotton](#) Dec 12, 2016 at 15:05
-

302

Here is an example that utilizes `fread` from `data.table` 1.8.7

The examples come from the help page to `fread`, with the timings on my windows XP Core 2 duo E8400.

```
library(data.table)
# Demo speedup
n=1e6
DT = data.table( a=sample(1:1000,n,replace=TRUE),
                 b=sample(1:1000,n,replace=TRUE),
                 c=rnorm(n),
                 d=sample(c("foo", "bar", "baz", "qux", "quux"),n,replace=TRUE),
                 e=rnorm(n),
                 f=sample(1:1000,n,replace=TRUE) )
DT[2,b:=NA_integer_]
DT[4,c:=NA_real_]
DT[3,d:=NA_character_]
DT[5,d:=""]
DT[2,e:=+Inf]
DT[3,e:=-Inf]
```

standard read.table

```
write.table(DT,"test.csv",sep=" ",row.names=FALSE,quote=FALSE)
cat("File size (MB):",round(file.info("test.csv")$size/1024^2),"\\n")
## File size (MB): 51

system.time(DF1 <- read.csv("test.csv",stringsAsFactors=FALSE))
##   user  system elapsed
## 24.71   0.15   25.42
# second run will be faster
system.time(DF1 <- read.csv("test.csv",stringsAsFactors=FALSE))
##   user  system elapsed
## 17.85   0.07   17.98
```

optimized read.table

```
system.time(DF2 <- read.table("test.csv",header=TRUE,sep=" ",quote="",
                              stringsAsFactors=FALSE,comment.char="",nrows=n,
                              colClasses=c("integer","integer","numeric",
                                             "character","numeric","integer"))))

##   user  system elapsed
## 10.20   0.03   10.32
```

fread

```
require(data.table)
system.time(DT <- fread("test.csv"))
##   user  system elapsed
##   3.12   0.01   3.22
```

sqldf

```
require(sqldf)

system.time(SQLDF <- read.csv.sql("test.csv",dbname=NULL))

##    user  system elapsed
##  12.49    0.09   12.69

# sqldf as on SO

f <- file("test.csv")
system.time(SQLf <- sqldf("select * from f", dbname = tempfile(), file.format =
list(header = T, row.names = F)))

##    user  system elapsed
##  10.21    0.47   10.73
```

ff / ffd

```
require(ff)

system.time(FFDF <- read.csv.ffdf(file="test.csv",nrows=n))
##    user  system elapsed
##  10.85    0.10   10.99
```

In summary:

##	user	system	elapsed	Method
##	24.71	0.15	25.42	read.csv (first time)
##	17.85	0.07	17.98	read.csv (second time)
##	10.20	0.03	10.32	Optimized read.table
##	3.12	0.01	3.22	fread
##	12.49	0.09	12.69	sqldf
##	10.21	0.47	10.73	sqldf on SO
##	10.85	0.10	10.99	ffdf

Share Improve this answer Follow

edited Jun 20, 2020 at 9:12



answered Feb 25, 2013 at 1:07



mnel

113k 27 264 253

51 Great answer, and the benchmarking holds in other contexts. Just read in a 4GB file in well under a minute with fread . Had tried reading it in with the base R functions and it took about 15 hours. – Ari B. Friedman Sep 12, 2013 at 13:55

1 my benchmark suggest even greater speed advantages for read.csv in data.table. note that data.table is not standard R, but (sadly) "just" nicely shared by its creators on CRAN. it is not even deemed standard enough to make the common R package list, much less qualify as a replacement for data frames. it has a lot of advantages, but also some very counterintuitive aspects. you may want to use as.data.frame(fread.csv("test.csv")) with the package to get back into the standard R data frame world. – ivo Welch Mar 17, 2015 at 2:31

3 @mnel could you please re-run the benchmark and include readr ? – jangorecki Dec 9, 2015 at 2:33

2 Second @jangorecki. Also, given fread has some real competitors now, could be useful to add benchmarks for optimized fread usage -- specifying colClasses , etc. – MichaelChirico Feb 7, 2016 at 20:23

1 @jangorecki @ MichaelChirico the code given is entirely reproducible so it is straight forward to simulate readr

I didn't see this question initially and asked a similar question a few days later. I am going to take my previous question down, but I thought I'd add an answer here to explain how I used `sqldf()` to do this.

There's been [little bit of discussion](#) as to the best way to import 2GB or more of text data into an R data frame. Yesterday I wrote a [blog post](#) about using `sqldf()` to import the data into SQLite as a staging area, and then sucking it from SQLite into R. This works really well for me. I was able to pull in 2GB (3 columns, 40mm rows) of data in < 5 minutes. By contrast, the `read.csv` command ran all night and never completed.

Here's my test code:

Set up the test data:

```
bigdf <- data.frame(dim=sample(letters, replace=T, 4e7), fact1=rnorm(4e7),
  fact2=rnorm(4e7, 20, 50))
write.csv(bigdf, 'bigdf.csv', quote = F)
```

I restarted R before running the following import routine:

```
library(sqldf)
f <- file("bigdf.csv")
system.time(bigdf <- sqldf("select * from f", dbname = tempfile(), file.format =
  list(header = T, row.names = F)))
```

I let the following line run all night but it never completed:

```
system.time(big.df <- read.csv('bigdf.csv'))
```

Share Improve this answer Follow

edited Nov 21, 2011 at 21:36



bnjmn

4,498 4 36 52

answered Nov 30, 2009 at 15:48



JD Long

59.3k 57 201 293

- 1 Hi. How woul you use it as an input for other packages such as zoo, designed to be used with all data simultanously? – [skan](#) Mar 27, 2013 at 1:15

@skan the end object is a data frame. So you have to convert it to a zoo object in order to use it with zoo. Look at the examples in the zoo docs for illustrations. – [JD Long](#) Apr 2, 2013 at 12:48

@JD Long. Hi, the problem is that when you convert it to a zoo object it tries to fit it on the memory. If it's too big it produces an error. And if the result of the zoo object (for example an aggregation of two series) is also too it would need to be a sql or ff object too. – [skan](#) Apr 3, 2013 at 14:03

I don't know what's wrong with sqldf. I've created a simple 1GB file on disk (with 2 numerical columns) and used `DTSQL <- read.csv.sql("f2.txt", dbname=tempfile())` and it tries to load the whole data on memory. Tomorrow I'll try ff and revoscaler instead. – [skan](#) Apr 4, 2013 at 0:38

- 1 @what m is thousand so mm is thousand thousand, or million. I probably should have capitalized it as MM. But I find that just about any million abbreviation can be confusing to someone if you have a diverse enough audience. In my attempt to be overly verbose, I'm sorry that I made it more confusing! [accountingcoach.com/blog/what-does-m-and-mm-stand-for](#) – [JD Long](#) Jan 4, 2016 at 19:13

81

Strangely, no one answered the bottom part of the question for years even though this is an important one -- `data.frame`s are simply lists with the right attributes, so if you have large data you don't want to use `as.data.frame` or similar for a list. It's much faster to simply "turn" a list into a data frame in-place:

```
attr(df, "row.names") <- .set_row_names(length(df[[1]]))
class(df) <- "data.frame"
```

This makes no copy of the data so it's immediate (unlike all other methods). It assumes that you have already set `names()` on the list accordingly.

[As for loading large data into R -- personally, I dump them by column into binary files and use `readBin()` - that is by far the fastest method (other than `mmap`) and is only limited by the disk speed. Parsing ASCII files is inherently slow (even in C) compared to binary data.]

Share Improve this answer Follow

edited Feb 24, 2013 at 21:42

answered Dec 20, 2012 at 4:01



Simon Urbanek

13.8k 45 45

-
- 6 Using `tracmem` suggests that `attr<-` and `class<-` make copies internally. `bit::setattr` or `data.table::setattr` will not. – [mnel](#) Dec 20, 2012 at 4:20

 - 6 Maybe you used the wrong order? There is no copy if you use `df=scan(...); names(df)=...; attr...; class...` - see `tracmem()` (tested in R 2.15.2) – [Simon Urbanek](#) Dec 20, 2012 at 4:31

 - 6 Can you elaborate on how you dump the large data by column into binary files? – [dabsingh](#) Sep 14, 2016 at 23:32

 - would you mind providing also an example please? – [Angelo](#) Jan 21, 2022 at 16:21

This was previously [asked on R-Help](#), so that's worth reviewing.

33

One suggestion there was to use `readChar()` and then do string manipulation on the result with `strsplit()` and `substr()`. You can see the logic involved in `readChar` is much less than `read.table`.

I don't know if memory is an issue here, but you might also [want to take a look at the HadoopStreaming package](#). This [uses Hadoop](#), which is a MapReduce framework designed for dealing with large data sets. For this, you would use the `hsTableReader` function. This is an example (but it has a learning curve to learn Hadoop):

```
str <- "key1\t3.9\nkey1\t8.9\nkey1\t1.2\nkey1\t3.9\nkey1\t8.9\nkey1\t1.2\nkey2\n\t9.9\nkey2\n"
cat(str)
cols = list(key='',val=0)
con <- textConnection(str, open = "r")
hsTableReader(con,cols,chunkSize=6,FUN=print,ignoreKey=TRUE)
close(con)
```

The basic idea here is to break the data import into chunks. You could even go so far as to use one of the parallel frameworks (e.g. `snow`) and run the data import in parallel by segmenting the file, but most likely for large data sets that won't help since you will run into memory constraints, which is why map-reduce is a better approach.

Share Improve this answer Follow

edited Oct 11, 2011 at 9:34



Roman Luštrik

69.2k 24 152 195

answered Nov 13, 2009 at 15:18



Shane

98.2k 35 223 217

I just did a quick test and `readChar` does seem to be much faster than even `readLines` for some inexplicable reason. However, it is still slow as `sin` compared to a simple C test. At the simple task of reading 100 megs, R is about 5 - 10x slower than C – [Jonathan Chang](#) Nov 13, 2009 at 20:12

1 Don't understand your point. The point of Hadoop is to handle very large data, which is what the question was about. – [Shane](#) Jan 23, 2010 at 1:33

1 Despite the name, `hsTableReader` doesn't have anything to do with Hadoop per se, it's for processing large data in pieces. It reads from `con`, a chunk of rows at a time, and passes each chunk as a `data.frame` to `FUN` for processing. With `ignoreKey=FALSE`, it does some extra grouping by key (the entry in the first column), which is relevant to Map/Reduce approaches. – [DavidR](#) Mar 4, 2013 at 22:46

Hi. How would you use this Hadoop data as an input for other packages such as `zoo`, designed to be used with all data simultaneously? – [skan](#) Mar 27, 2013 at 1:15

An alternative is to use the `vroom` package. Now on CRAN. `vroom` doesn't load the entire file, it indexes where each record is located, and is read later when you use it.

Only pay for what you use.



See [Introduction to vroom](#), [Get started with vroom](#) and the [vroom benchmarks](#).

The basic overview is that the initial read of a huge file, will be much faster, and subsequent modifications to the data may be slightly slower. So depending on what your use is, it could be the best option.

See a simplified example from [vroom benchmarks](#) below, the key parts to see is the super fast read times, but slightly slower operations like aggregate etc..

package	read	print	sample	filter	aggregate	total
read.delim	1m	21.5s	1ms	315ms	764ms	1m 22.6s
readr	33.1s	90ms	2ms	202ms	825ms	34.2s
data.table	15.7s	13ms	1ms	129ms	394ms	16.3s
vroom (altrep) dplyr	1.7s	89ms	1.7s	1.3s	1.9s	6.7s

Share Improve this answer Follow

edited May 11, 2019 at 10:20

answered May 7, 2019 at 21:00



Jaap

80.4k 34 181 192



Hector Haffenden

1,350 10 25



I am reading data very quickly using the new `arrow` package. It appears to be in a fairly early stage.

10



Specifically, I am using the **parquet** columnar format. This converts back to a `data.frame` in R, but you can get even deeper speedups if you do not. This format is convenient as it can be used from Python as well.



My main use case for this is on a fairly restrained RShiny server. For these reasons, I prefer to keep data attached to the Apps (i.e., out of SQL), and therefore require small file size as well as speed.



This linked article provides benchmarking and a good overview. I have quoted some interesting points below.

<https://ursalabs.org/blog/2019-10-columnar-perf/>

File Size

That is, the Parquet file is half as big as even the gzipped CSV. One of the reasons that the Parquet file is so small is because of dictionary-encoding (also called “dictionary compression”). Dictionary compression can yield substantially better compression than using a general purpose bytes compressor like LZ4 or ZSTD (which are used in the FST format). Parquet was designed to produce very small files that are fast to read.

Read Speed

When controlling by output type (e.g. comparing all R `data.frame` outputs with each other) we see the the performance of Parquet, Feather, and FST falls within a relatively small margin of each other. The same is true of the pandas.DataFrame outputs. `data.table::fread` is impressively competitive with the 1.5 GB file size but lags the others on the 2.5 GB CSV.

Independent Test

I performed some independent benchmarking on a simulated dataset of 1,000,000 rows. Basically I shuffled a bunch of things around to attempt to challenge the compression. Also I added a short text field of random words and two simulated factors.

Data

```
library(dplyr)
library(tibble)
library(OpenRepGrid)

n <- 1000000

set.seed(1234)
some_levels1 <- sapply(1:10, function(x) paste(LETTERS[sample(1:26, size = sample(3:8,
1), replace = TRUE)], collapse = ""))
some_levels2 <- sapply(1:65, function(x) paste(LETTERS[sample(1:26, size = sample(5:16,
1), replace = TRUE)], collapse = ""))

test_data <- mtcars %>%
  rownames_to_column() %>%
```

```
rownames_to_column(), %>%
  sample_n(n, replace = TRUE) %>%
  mutate_all(~ sample(., length(.))) %>%
  mutate(factor1 = sample(some_levels1, n, replace = TRUE),
         factor2 = sample(some_levels2, n, replace = TRUE),
         text = randomSentences(n, sample(3:8, n, replace = TRUE))
  )
```

Read and Write

Writing the data is easy.

```
library(arrow)

write_parquet(test_data , "test_data.parquet")

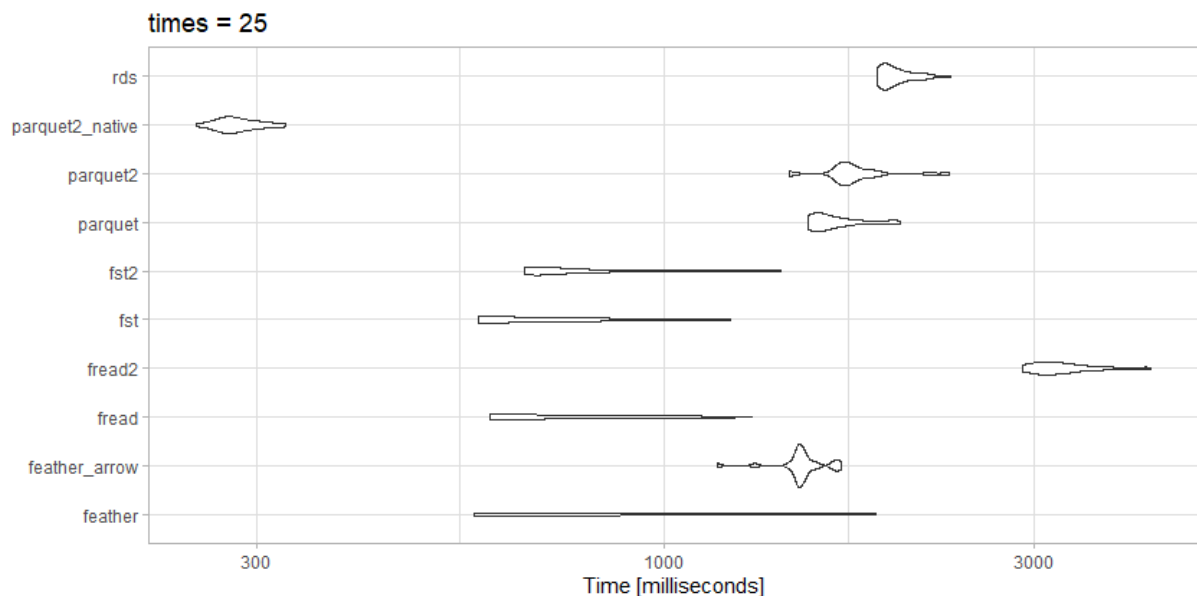
# you can also mess with the compression
write_parquet(test_data, "test_data2.parquet", compress = "gzip", compression_level =
9)
```

Reading the data is also easy.

```
read_parquet("test_data.parquet")

# this option will result in lightning fast reads, but in a different format.
read_parquet("test_data2.parquet", as_data_frame = FALSE)
```

I tested reading this data against a few of the competing options, and did get slightly different results than with the article above, which is expected.



This file is nowhere near as large as the benchmark article, so maybe that is the difference.

Tests

- **rds:** test_data.rds (20.3 MB)
- **parquet2_native:** (14.9 MB with higher compression and `as_data_frame = FALSE`)
- **parquet2:** test_data2.parquet (14.9 MB with higher compression)
- **parquet:** test_data.parquet (40.7 MB)

- **fst2:** test_data2.fst (27.9 MB with higher compression)
- **fst:** test_data.fst (76.8 MB)
- **fread2:** test_data.csv.gz (23.6MB)
- **fread:** test_data.csv (98.7MB)
- **feather_arrow:** test_data.feather (157.2 MB read with `arrow`)
- **feather:** test_data.feather (157.2 MB read with `feather`)

Observations

For this particular file, `fread` is actually very fast. I like the small file size from the highly compressed `parquet2` test. I may invest the time to work with the native data format rather than a `data.frame` if I really need the speed up.

Here `fst` is also a great choice. I would either use the highly compressed `fst` format or the highly compressed `parquet` depending on if I needed the speed or file size trade off.

Share Improve this answer Follow

answered Nov 12, 2019 at 18:31
user10917479



6



A minor additional points worth mentioning. If you have a very large file you can on the fly calculate the number of rows (if no header) using (where `bedGraph` is the name of your file in your working directory):

```
>numRow=as.integer(system(paste("wc -l", bedGraph, "| sed 's/^0-9.*\\([0-9.*\\).*/\\1/'", intern=T))
```

You can then use that either in `read.csv` , `read.table` ...

```
>system.time((BG=read.table(bedGraph, nrow=numRow, col.names=c('chr', 'start', 'end', 'score'), colClasses=c('character', rep('integer',3)))))
user system elapsed
25.877 0.887 26.752
>object.size(BG)
203949432 bytes
```

Share Improve this answer Follow

answered Nov 28, 2013 at 17:20



Stephen Henderson
6,270 3 26 33

6

Often times I think it is just good practice to keep larger databases inside a database (e.g. Postgres). I don't use anything too much larger than (nrow * ncol) ncell = 10M, which is pretty small; but I often find I want R to create and hold memory intensive graphs only while I query from multiple databases. In the future of 32 GB laptops, some of these types of memory problems will disappear. But the allure of using a database to hold the data and then using R's memory for the resulting query results and graphs still may be useful. Some advantages are:

(1) The data stays loaded in your database. You simply reconnect in pgadmin to the databases you want when you turn your laptop back on.

(2) It is true R can do many more nifty statistical and graphing operations than SQL. But I think SQL is better designed to query large amounts of data than R.

```
# Looking at Voter/Registrant Age by Decade
```

```
library(RPostgreSQL);library(lattice)
```

```
con <- dbConnect(PostgreSQL(), user= "postgres", password="password",
                  port="2345", host="localhost", dbname="WC2014_08_01_2014")
```

```
Decade_BD_1980_42 <- dbGetQuery(con,"Select PrecinctID,Count(PrecinctID),extract(DECADE
from Birthdate) from voterdb where extract(DECADE from Birthdate)::numeric > 198 and
PrecinctID in (Select * from LD42) Group By PrecinctID,date_part Order by Count DESC;")
```

```
Decade_RD_1980_42 <- dbGetQuery(con,"Select PrecinctID,Count(PrecinctID),extract(DECADE
from RegistrationDate) from voterdb where extract(DECADE from
RegistrationDate)::numeric > 198 and PrecinctID in (Select * from LD42) Group By
PrecinctID,date_part Order by Count DESC;")
```

```
with(Decade_BD_1980_42,(barchart(~count | as.factor(precinctid))));
mtext("42LD Birthdays later than 1980 by Precinct",side=1,line=0)
```

```
with(Decade_RD_1980_42,(barchart(~count | as.factor(precinctid))));
mtext("42LD Registration Dates later than 1980 by Precinct",side=1,line=0)
```

Share Improve this answer Follow

edited Aug 24, 2014 at 15:54



IRTFM

258k

21

362

485

answered Aug 22, 2014 at 16:13



rferrisx

1,568

2

12

14

DuckDB is a relatively new open-source analytical database available now on CRAN. Very small package while supporting PostGres like SQL commands. It also supports querying parquet format files with SQL commands.

– San Aug 7, 2021 at 16:25

I wanted to contribute Spark-based solution in the simplest form:

1

```
# Test Data -----  
  
set.seed(123)  
bigdf <-  
  data.frame(  
    dim = sample(letters, replace = T, 4e7),  
    fact1 = rnorm(4e7),  
    fact2 = rnorm(4e7, 20, 50)  
  )  
tmp_csv <- fs::file_temp(pattern = "big_df", ext = ".csv")  
readr::write_csv(x = bigdf, file = tmp_csv)  
  
# Spark -----  
  
# Installing if needed  
# sparklyr::spark_available_versions()  
# sparklyr::spark_install()  
  
library("sparklyr")  
sc <- spark_connect(master = "local")  
  
# Uploading CSV  
system.time(tbl_big_df <- spark_read_csv(sc = sc, path = tmp_csv))
```

Spark generated fairly OK results:

```
>> system.time(tbl_big_df <- spark_read_csv(sc = sc, path = tmp_csv))  
   user  system elapsed  
 0.278   0.034  11.747
```

This was tested on MacBook Pro with 32GB ram.

Remarks

Spark, *usually* shouldn't be able to "win" against packages optimised for speed. Nevertheless, I wanted to contribute an answer using Spark:

- For some of the comments and answers where process didn't work using Spark may be a viable alternative
- In a long-run, hammering as much data as possible into `data.frame` may prove problematic later on, when other operations are attempted on that object and hit the performance envelope of architecture

I think that for questions like that, where the task is to handle 1e7 or more rows Spark should be given considerations. Even if it may be possible to "hammer in" that data into a single `data.frame` it's just doesn't feel right. Likely that object will be difficult to work with and create problems when deploying models, etc.

Share Improve this answer Follow

answered Dec 18, 2021 at 20:19



Konrad

17.5k

16

103

164



Instead of the conventional `read.table` I feel `fread` is a faster function. Specifying additional attributes like select only the required columns, specifying colclasses and string as factors will reduce the time take to import the file.

```
data_frame <-  
fread("filename.csv", sep=";", header=FALSE, stringsAsFactors=FALSE, select=c(1,4,5,6,7), colC
```

Share Improve this answer Follow

answered Apr 18, 2015 at 7:22



Aayush Agrawal

184 1 6



I've tried all above and `[readr][1]` made the best job. I have only 8gb RAM

Loop for 20 files, 5gb each, 7 columns:

```
read_fwf(arquivos[i], col_types = "ccccccc", fwf_cols(cnpj = c(4,17), nome = c(19,168),  
cpf = c(169,183), fantasia = c(169,223), sit.cadastral = c(224,225), dt.sitcadastral =  
c(226,233), cnae = c(376,382)))
```

Share Improve this answer Follow

edited Jan 2, 2020 at 18:47

answered Dec 29, 2019 at 0:41



Bruno Gomes

85 8

 **Highly active question.** Earn 10 reputation (not counting the [association bonus](#)) in order to answer this question. The reputation requirement helps protect this question from spam and non-answer activity.